

Machine Learning for IoT

Lab 5 – Communication

HowTo 1: JSON in Python

JSON (JavaScript Object Notation) is a lightweight data-interchange format that is used for exchanging data between different systems. JSON is commonly used for transmitting data over a network connection (e.g., between a server and a web application) and for storing data in a file or database.

JSON is a widely used format because it is easy to read and write, and it is supported by many programming languages and tools. For example, you can use the *json* module in Python to encode and decode JSON data.

`json.loads(json_string)` : convert a JSON string to a Python object

```
string = '{"name": "Tony", "surname": "Stark"}'
obj = json.loads(string)
```

`json.dumps(object)` : convert a Python object to a JSON string

```
obj = {"num1": 12, "num2": 34}
string = json.dumps(obj)
```

HowTo 2: Setup REST Client in VSCode

1. Open VSCode and install *REST Client* extension by Huachao Mao.
2. Create a new file and save it with the *.http* extension.
3. In the file, specify the base URL of the API and the endpoint that you want to test. For example:

```
GET https://jsonplaceholder.typicode.com/users
```

Exercise 1: MQTT in Python with PAHO

- 1.1 On the RPI, develop an MQTT publisher to send:

- every 5 seconds the date and time in the format dd-mm-yyyy hh:mm:ss.
- every 10 seconds the timestamp.

Use hierarchical topics for the two messages, with your student ID as the first topic level (e.g., *s001122/datetime* and *s001122/timestamp*)

Use the message broker provided by eclipse ([mqtt.eclipseprojects.io](https://eclipseprojects.io) at port 1883).

- 1.2 In Deepnote, develop a second MQTT subscriber to receive only the messages about the timestamp and print the information in a user-friendly format.

Exercise 2: REST Client with VSCode

In VSCode, create a new file named *lab5_ex2.http* to develop a REST Client that retrieve information related to cryptocurrencies, such as prices, market data, trading information, and more from a third-party service, the Coinbase Data API (see the documentation: <https://docs.cloud.coinbase.com/sign-in-with-coinbase/docs/api-currencies>).

In the file, define HTTP requests to retrieve the following information:

- Get a list of all available currencies.
- Get the exchange rates for the EUR currency.
- Get the price in EUR to buy one bitcoin (BTC).
- Get the price in EUR to sell one bitcoin (BTC).

Exercise 3: REST Client with Python

In VSCode, write a Python script named *lab5_ex3.py* to retrieve information from the Coinbase Data API using the *requests* package. The script should be run from the command line interface and should take as input a single argument called *--currency* that specifies the currency code.

Then, it should print the buy and sell price for one bitcoin in human readably format. If the currency does not exist, print an error message.

Suggestion: check the official Python documentation of the *requests* package at: <https://requests.readthedocs.io/en/latest/user/quickstart/#make-a-request>

Example:

```
python lab5_ex3.py --currency EUR
```

Output:

```
The buy price in EUR for one BTC is: 16367.88
The sell price in EUR for one BTC is: 16203.09
```

Example:

```
python lab5_ex3.py --currency AAA
```

Output:

```
Error: AAA currency not found.
```

Exercise 4: REST Server with CherryPy

4.1 Develop a sensor timeseries management service using *cherryPy* as web framework and Redis as database.

The service should provide a simple REST API to create/delete timeseries of T/H sensor nodes and retrieve information about the collected data. The API enables the user to add sensors, retrieve the list of monitored sensors, update their timeseries settings, and delete sensors. The API must be compliant with the following specifications:

RESOURCES

Resource: Sensor

Parameter	Description
mac_address	string, MAC address of the sensor node.
t_samples	int, number of temperature samples.
t_retention	int, temperature retention period in ms.
h_samples	int, number of humidity samples.
h_retention	int, humidity retention period in ms.

Example:

```
{
  "mac_address": "0xf0b61e0bfe09",
  "t_samples": 10,
  "t_retention": 86400000,
  "h_samples": 10,
  "h_retention": 86400000,
}
```

ENDPOINTS

Endpoint /status

- **GET /status**

Description: Return the status of the REST Server.

Path Parameters: N/A

Query Parameters: N/A

Response Status Code:

- 200 – OK: Everything worked as expected.

Response Schema:

Parameter	Description
status	string, equal to “online” if the REST Server is online.

Response Example:

```
{
  status: "online"
}
```

Endpoint /sensors

- **GET /sensors**

Description: Get the list of monitored sensors. This endpoint allows you to filter sensors based on the number of temperature/humidity samples.

Path Parameters: N/A

Query Parameters:

Parameter	Description
min_t_samples	int (optional), an integer representing the minimum number of temperature samples. If specified, return sensors with a number of temperature samples greater or equal than the specified value. If not provided, sensors with any number of temperature samples will be returned.
min_h_samples	int (optional), an integer representing the minimum number of humidity samples. If specified, return sensors with a number of humidity samples greater or equal than the specified value. If not provided, sensors with any number of humidity samples will be returned.

Response Status Code:

- 200 – OK: Everything worked as expected.

Response Schema: Dictionary with the following keys:

- **sensors:** List of **Sensor** resources
- **count:** int, number of returned sensors.

Response Example:

```
{
  "sensors": [
    {
      "mac_address": "0xf0b61e0bfe09",
      "t_samples": 10,
      "t_retention": 86400000,
      "h_samples": 10,
      "h_retention": 86400000,
    },
    {
      "mac_address": "0xf0d34d1cfac7",
      "t_samples": 25,
      "t_retention": 86400000,
      "h_samples": 25,
      "h_retention": 86400000,
    },
  ],
  "count": 2
}
```

- **POST /sensors**

Description: Add a sensor and create Redis timeseries to store temperature and humidity with a retention period of 1 day.

Path Parameters: N/A

Query Parameters: N/A

Body Parameters:

Parameter	Description
mac_address	string (required), MAC address of the sensor.

Response Status Code:

- 200 – OK: Everything worked as expected.
- 400 – Bad Request: Missing MAC address in the request body.
- 409 – Conflict: Sensor already exists.

Response Schema: N/A

Endpoint /sensor/{id}

- **GET /sensor/{mac_address}**

Description: Retrieve the specified sensor resource.

Path Parameters:

Parameter	Description
mac_address	string (required), the MAC address of the sensor to retrieve

Query Parameters: N/A

Response Status Code:

- 200 – OK: Everything worked as expected.
- 400 – Bad Request: Missing MAC address in the request parameters.
- 404 – Not Found: MAC address not found in the database.

Response Schema: Sensor

Response Example:

```
{
  "mac_address": "0xf0b61e0bfe09",
  "t_samples": 10,
  "t_retention": 86400000,
  "h_samples": 10,
  "h_retention": 86400000,
}
```

- **PUT /sensor/{mac_address}**

Description: Update temperature and humidity retention periods for the specified sensor.

Path Parameters:

Parameter	Description
mac_address	string (required), the MAC address of the sensor to update

Query Parameters: N/A

Body Parameters:

Parameter	Description
t_retention	int (required), temperature retention period in ms.
h_retention	int (required), humidity retention period in ms.

Response Status Code:

- 200 – OK: Everything worked as expected.
- 400 – Bad Request: Missing MAC address in the request parameters.
- 400 – Bad Request: Missing temperature retention period in the request body.
- 400 – Bad Request: Missing humidity retention period in the request body.
- 404 – Not Found: MAC address not available in the database.

Response Schema: N/A

- **DELETE /sensor/{mac_address}**

Description: Delete the timeseries of the specified sensor from the database.

Path Parameters:

Parameter	Description
mac_address	string (required), MAC address of the sensor to update.

Query Parameters: N/A

Response Status Code:

- 200 – OK: Everything worked as expected.
- 400 – Bad Request: Missing MAC address in the request parameters.
- 404 – Not Found: MAC address not found in the database.

Response Schema: N/A

4.2 Deploy the REST API on Deepnote.

- In the *Machine* tab, set “Incoming connections” to ON. Copy the tunnelling link.
- Run the REST API notebook.
- In the VSCode and Python REST clients, use the tunnelling link of Deepnote as the API host.

4.3 Test the API with a VSCode Client. The test must sequentially execute the following actions:

- Check the server status.
- Add one sensor.
- Print the list of sensors.
- Print the list of sensors with more than 10 temperature records.
- Modify the temperature retention period of the created sensor to 2 days.
- Delete the sensor timeseries.

4.4 Test the API with a Python Client. Repeat the same actions of 4.3.